

Stage 3.1 — Creating a proper Settings object

Great. This is the right next step. You already have YAML loading, automatic discovery, merging, and settings.py working. Now we'll improve the design.

Stage 3.1 — Creating a proper Settings object

Currently your settings.py does this:

```
LLM_MODEL = _CONFIG["llm"]["model"]
EMBEDDING_MODEL = _CONFIG["embedding"]["model"]
TOP_K = _CONFIG["retrieval"]["top_k"]
```

It works, but we're still dealing with flat variables extracted from a raw dictionary.

We want to move toward:

```
settings.llm.model
settings.embedding.model
settings.retrieval.top_k
settings.chunking.chunk_size
```

This gives us a much cleaner configuration API.

1. Our target architecture

We're going from:

```
YAML
↓
Dictionary
↓
_CONFIG["llm"]["model"]
```

to:

```
YAML
↓
Dictionary
↓
Settings object
↓
settings.llm.model
```

Eventually this will become:



2. Why do we need a Settings object?

Consider your current approach:

```
LLM_MODEL = _CONFIG["llm"]["model"]
LLM_TEMPERATURE = _CONFIG["llm"]["temperature"]

EMBEDDING_MODEL = _CONFIG["embedding"]["model"]

TOP_K = _CONFIG["retrieval"]["top_k"]
```

Imagine you eventually have 50 configuration values.

Your settings.py could become huge.

Instead:

```
settings.llm.model
settings.llm.temperature

settings.embedding.model

settings.retrieval.top_k
settings.retrieval.reranking

settings.chunking.chunk_size
settings.chunking.chunk_overlap
```

The configuration becomes organized according to its domain.

That's much closer to how enterprise applications are designed.

3. Create settings.py

We're going to change our current settings.py.

FILE: FILE SRC/SETTINGS.PY

```

from pathlib import Path
from typing import Any

from .config_loader import load_all_configs

class Settings:
    """
    Central configuration object for the application.
    """

    def __init__(self, config: dict[str, Any]):

        # Store the complete configuration dictionary.
        self._config = config

        # Application configuration
        self.application = config["application"]

        # LLM configuration
        self.llm = config["llm"]

        # Embedding configuration
        self.embedding = config["embedding"]

        # Vector database configuration
        self.vector_database = config["vector_database"]

        # Retrieval configuration
        self.retrieval = config["retrieval"]

        # Chunking configuration
        self.chunking = config["chunking"]

        # Logging configuration
        self.logging = config["logging"]

    # -----
    # Locate the project root directory
    # -----

    BASE_DIR = Path(__file__).resolve().parent.parent

    # -----
    # Locate the YAML configuration directory
    # -----

    CONFIG_DIR = BASE_DIR / "config"

    # -----
    # Load all YAML configuration files
    # -----

```

```
_CONFIG = load_all_configs(CONFIG_DIR)

# -----
# Create the application's Settings object
# -----

settings = Settings(_CONFIG)
```

Now we have a real:

```
settings
```

object.

4. Test it

Modify:

FILE: FILE MAIN.PY

```
from src.settings import settings

print("Application:", settings.application["name"])

print("LLM Model:", settings.llm["model"])

print("Embedding Model:", settings.embedding["model"])

print(
    "Collection:",
    settings.vector_database["collection_name"]
)

print("Top K:", settings.retrieval["top_k"])

print("Chunk Size:", settings.chunking["chunk_size"])

print("Chunk Overlap:", settings.chunking["chunk_overlap"])
```

Run:

```
python main.py
```

You should get:

```
Application: Enterprise RAG application
LLM Model: llama-3.1-8b-instant
Embedding Model: all-MiniLM-L6-v2
Collection: enterprise_knowledge_base
Top K: 5
Chunk Size: 1000
Chunk Overlap: 200
```

5. But wait...

You may notice that we're still doing:

```
settings.llm["model"]
```

instead of:

```
settings.llm.model
```

That's because:

```
settings.llm
```

is still a dictionary.

For example:

```
settings.llm
```

currently contains:

```
{
  "provider": "Groq",
  "model": "llama-3.1-8b-instant",
  "temperature": 0
}
```

So we still need:

```
["model"]
```

We haven't reached our final design yet.

6. Let's create nested configuration classes

This is where things get interesting.

We can create:

```
class LLMSettings:
```

which represents:

```
llm:  
  provider: Groq  
  model: llama-3.1-8b-instant  
  temperature: 0
```

Similarly:

```
class EmbeddingSettings:
```

represents:

```
embedding:  
  provider: sentence-transformers  
  model: all-MiniLM-L6-v2
```

And:

```
class RetrievalSettings:
```

represents:

```
retrieval:  
  top_k: 5  
  reranking: false
```

7. Update settings.py

Let's now make our first proper version.

FILE: FILE SRC/SETTINGS.PY

```

from pathlib import Path
from typing import Any

from .config_loader import load_all_configs

# =====
# LLM configuration
# =====

class LLMSettings:

    def __init__(self, config: dict[str, Any]):

        # Store the LLM provider.
        self.provider = config["provider"]

        # Store the LLM model name.
        self.model = config["model"]

        # Store the temperature used by the LLM.
        self.temperature = config["temperature"]

# =====
# Embedding configuration
# =====

class EmbeddingSettings:

    def __init__(self, config: dict[str, Any]):

        # Embedding provider.
        self.provider = config["provider"]

        # Embedding model.
        self.model = config["model"]

# =====
# Vector database configuration
# =====

class VectorDatabaseSettings:

    def __init__(self, config: dict[str, Any]):

        # Vector database provider.
        self.provider = config["provider"]

        # Database directory.
        self.directory = config["directory"]

        # Collection name.
        self.collection_name = config["collection_name"]

```

```

# =====
# Retrieval configuration
# =====

class RetrievalSettings:

    def __init__(self, config: dict[str, Any]):

        # Number of documents to retrieve.
        self.top_k = config["top_k"]

        # Whether reranking is enabled.
        self.reranking = config["reranking"]

# =====
# Chunking configuration
# =====

class ChunkingSettings:

    def __init__(self, config: dict[str, Any]):

        # Chunking strategy.
        self.strategy = config["strategy"]

        # Number of characters/tokens used per chunk,
        # depending on the splitter implementation.
        self.chunk_size = config["chunk_size"]

        # Number of overlapping characters/tokens.
        self.chunk_overlap = config["chunk_overlap"]

# =====
# Application configuration
# =====

class ApplicationSettings:

    def __init__(self, config: dict[str, Any]):

        # Application name.
        self.name = config["name"]

        # Application version.
        self.version = config["version"]

        # Current environment.
        self.environment = config["environment"]

# =====
# Logging configuration
# =====

```



```

class LoggingSettings:

    def __init__(self, config: dict[str, Any]):

        # Logging level.
        self.level = config["level"]

        # Logging output file.
        self.file = config["file"]

# =====
# Main Settings object
# =====

class Settings:

    def __init__(self, config: dict[str, Any]):

        # Convert each YAML section into a dedicated
        # configuration object.

        self.application = ApplicationSettings(
            config["application"]
        )

        self.llm = LLMSettings(
            config["llm"]
        )

        self.embedding = EmbeddingSettings(
            config["embedding"]
        )

        self.vector_database = VectorDatabaseSettings(
            config["vector_database"]
        )

        self.retrieval = RetrievalSettings(
            config["retrieval"]
        )

        self.chunking = ChunkingSettings(
            config["chunking"]
        )

        self.logging = LoggingSettings(
            config["logging"]
        )

# =====
# Locate project root
# =====

```

```

BASE_DIR = Path(__file__).resolve().parent.parent

# =====
# Locate configuration directory
# =====

CONFIG_DIR = BASE_DIR / "config"

# =====
# Load all YAML files
# =====

_CONFIG = load_all_configs(CONFIG_DIR)

# =====
# Create one global Settings object
# =====

settings = Settings(_CONFIG)

```

This is a significant improvement.

8. Now change main.py

FILE: FILE MAIN.PY

```

from src.settings import settings

print("Application:", settings.application.name)

print("Version:", settings.application.version)

print("Environment:", settings.application.environment)

print("LLM Provider:", settings.llm.provider)

print("LLM Model:", settings.llm.model)

print("LLM Temperature:", settings.llm.temperature)

print("Embedding Provider:", settings.embedding.provider)

print("Embedding Model:", settings.embedding.model)

print(
    "Vector DB:",
    settings.vector_database.provider
)

print(
    "Collection:",
    settings.vector_database.collection_name
)

print("Top K:", settings.retrieval.top_k)

print(
    "Reranking:",
    settings.retrieval.reranking
)

print(
    "Chunk Size:",
    settings.chunking.chunk_size
)

print(
    "Chunk Overlap:",
    settings.chunking.chunk_overlap
)

```

Run:

```
python main.py
```

Expected:

```
Application: Enterprise RAG application
Version: 1.0
Environment: development
LLM Provider: Groq
LLM Model: llama-3.1-8b-instant
LLM Temperature: 0
Embedding Provider: sentence-transformers
Embedding Model: all-MiniLM-L6-v2
Vector DB: chroma
Collection: enterprise_knowledge_base
Top K: 5
Reranking: False
Chunk Size: 1000
Chunk Overlap: 200
```

9. Notice the difference

Before

```
config["llm"]["model"]
```

Then:

```
settings.llm["model"]
```

Now:

```
settings.llm.model
```

That's the transformation we wanted.

The configuration is now hierarchical:

```
settings
├── application
│   ├── name
│   ├── version
│   └── environment
├── llm
│   ├── provider
│   ├── model
│   └── temperature
├── embedding
│   ├── provider
│   └── model
├── vector_database
│   ├── provider
│   ├── directory
│   └── collection_name
├── retrieval
│   ├── top_k
│   └── reranking
├── chunking
│   ├── strategy
│   ├── chunk_size
│   └── chunk_overlap
└── logging
    ├── level
    └── file
```

This is much easier to understand.

10. This is now very close to your Enterprise RAG

Imagine your RAG's retriever.py needs top_k.
Instead of:

```
from settings import TOP_K
```

we can eventually use:

```
from src.settings import settings

documents = retriever.invoke(
    query,
    k=settings.retrieval.top_k
)
```

Your embedding component could use:

```
settings.embedding.model
```

Your LLM component:

```
settings.llm.model
```

Your Chroma component:

```
settings.vector_database.collection_name
```

Your chunking component:

```
settings.chunking.chunk_size
```

```
settings.chunking.chunk_overlap
```

So configuration becomes organized according to the component that owns it.

11. One important limitation remains

Our Settings object looks good:

```
settings.llm.model
```

But there's still a problem.

Suppose somebody changes llm.yaml to:

```
llm:
  provider: Groq
  model: llama-3.1-8b-instant
  temperature: hello
```

Our code will happily accept:

```
settings.llm.temperature
```

as:

```
"hello"
```

But an LLM expects a number.

Similarly:

```
retrieval:
```

```
  top_k: five
```

would be accepted as a string.

We currently have no type validation.

12. That's why Stage 3.2 is important

We'll introduce Pydantic.

Instead of manually writing:

```
class LLMSettings:

    def __init__(self, config):
        self.provider = config["provider"]
        self.model = config["model"]
        self.temperature = config["temperature"]
```

we'll eventually write something conceptually like:

```
class LLMSettings(BaseModel):
    provider: str
    model: str
    temperature: float
```

Then Pydantic can validate:

```
temperature: 0
```

✓ valid

but:

```
temperature: hello
```

✗ invalid

And:

```
top_k: 5
```

✓ valid

but:

```
top_k: five
```

✗ invalid

Your Stage 3.1 milestone

At this point you have progressed through:

```
Stage 1
Basic YAML
    ↓
Stage 2
PyYAML + pathlib
    ↓
Stage 2.4
Reusable load_yaml()
    ↓
Stage 2.5
Multiple YAML files
    ↓
Stage 2.6
Automatic YAML discovery
    ↓
Stage 3
YAML-based application configuration
    ↓
Stage 3.1 ← YOU ARE HERE
Settings object
    ↓
Stage 3.2
Pydantic validation
    ↓
Stage 3.3
YAML + .env + secrets
    ↓
Stage 3.4
Enterprise RAG configuration architecture
```

Don't move to Pydantic yet. First run the new Settings implementation and verify that you can access values using:

```
settings.llm.model
settings.embedding.model
settings.vector_database.collection_name
settings.retrieval.top_k
settings.chunking.chunk_size
```

Once that works, the next lesson will be Stage 3.2 — Pydantic Settings and configuration validation, where this becomes much closer to production-grade Python configuration.